

A LESSON IN RESEARCH METHODOLOGY

Deduction and Induction in Computer Science

Theoretical Models and Approaches

The three directions of inference a researcher can travel — deduction, induction, and abduction — and the theoretical models that make each of them possible in computing science.



Every Research Move Is an Inference

Lecture 2 asked how a claim is proved. This session asks the question underneath it: in which direction is the reasoning travelling, and what is that direction allowed to conclude?

- 1 There are only three directions available. Deduction moves from a general rule to a particular consequence. Induction moves from particulars to a general rule. Abduction moves from an observation to the hypothesis that would explain it.
- 2 Each direction has a fixed price. Deduction preserves truth but adds nothing new; induction adds new content but cannot preserve truth; abduction generates the hypotheses on which the other two go to work.
- 3 A CS paper normally uses all three, in sequence — and gets into trouble when it reports a conclusion drawn in one direction as though it had been reached in another.
- 4 This is also the machinery behind Unit I: a “research problem” is an abduction, and the “investigative approach” you choose determines which of the other two directions carries the argument.

THE PRICE OF EACH DIRECTION

No form of inference gives you both certainty and new information. You choose which one to give up.

A theorem is certain and tells you nothing you had not assumed. A benchmark result is informative and could be overturned tomorrow. Both are legitimate; conflating them is not.

Name the direction before you defend the claim.

Deduction, Induction, Abduction

Charles Sanders Peirce's three-way division, in the form CS actually uses it. Read the “form” row as the shape of the argument, stripped of content.

DIRECTION 1

Deduction

Rule + case $\Rightarrow$ result

MOVES FROM

A general rule to a particular consequence.

FORM

Sorted arrays are searchable in $O(\log n)$. A is sorted. So A is searchable in $O(\log n)$.

GUARANTEE

Truth-preserving. If the premises hold, the conclusion cannot fail.

IN CS

Type soundness, Hoare logic, complexity bounds, model checking.

DIRECTION 2

Induction

Case + result $\Rightarrow$ rule

MOVES FROM

Observed instances to a rule covering unobserved ones.

FORM

These 40 traces showed a 12% gain. So the method gains ~12% on such traces.

GUARANTEE

None. Ampliative — it says more than the premises.

IN CS

Machine learning, benchmarking, empirical SE, performance models.

DIRECTION 3

Abduction

Rule + result $\Rightarrow$ case

MOVES FROM

A surprising observation to a hypothesis explaining it.

FORM

Latency spikes hourly. An hourly cache flush would cause that. So suspect the flush.

GUARANTEE

None, and only as good as the rivals you considered.

IN CS

Debugging, fault localisation, root-cause analysis, problem framing.

Peirce's point was that the three are not competing accounts of reasoning — they are three distinct operations, and a working science needs all of them in rotation.

What Each Preserves, What Each Risks

The same three directions, read as a reviewer reads them: what does this step buy, and where can it break?

DIRECTION	ADDS NEW CONTENT?	PRESERVES TRUTH?	FAILS WHEN...	HOW IT IS DEFENDED
Deduction	No — it unpacks what the premises already entail.	Yes, absolutely, given the premises.	A premise is false, or the model it describes is not the system you deployed.	Expose the axioms. Show the inference steps. Argue the model applies.
Induction	Yes — the conclusion ranges beyond the sample.	No. The next observation may overturn it.	The sample is unrepresentative, or the distribution shifts after training.	Sampling design, controls, held-out data, effect size, a stated bias.
Abduction	Yes — it proposes an entirely new hypothesis.	No. Several hypotheses can explain one fact.	Rival explanations were never enumerated, so “best” means “only one considered”.	List the rivals. Derive a prediction that separates them. Then test it.

Notice the pattern: the only direction that guarantees its conclusion is the one that tells you nothing you had not already assumed.

“Mathematical Induction” Is Not Induction

A name collision that causes real confusion in CS writing — and the single most useful distinction in this session.

MATHEMATICAL INDUCTION

Prove $P(0)$. Prove $P(n) \Rightarrow P(n+1)$.
Conclude $\forall n \in \mathbb{N}, P(n)$.

The conclusion is already contained in the two premises together with the induction axiom. Nothing is added; nothing observed. It is a deductive rule that happens to be named after its inductive cousin.

Belongs to Lecture 2's mathematical mode. Only an error in the proof can defeat it.

EMPIRICAL INDUCTION

Observe $P(x_1) \dots P(x_m)$ on a sample.
Conclude P holds generally.

The conclusion says strictly more than the premises. It is supported, never entailed, and the next observation can overturn it however careful the sampling was.

Belongs to Lecture 2's empirical mode. This is the induction Hume worried about.

Why the collision matters

Write “we prove by induction that our approach generalises” and a reviewer cannot tell whether you are offering a theorem or a hope. The two carry completely different obligations.

A quick test

Ask what could overturn the conclusion. If only a flawed proof could, the reasoning is deductive. If a new observation could, it is inductive — regardless of the word used.

Structural induction too

Induction over terms, derivations, or data types is likewise deductive. It is the standard tool for proving properties of recursive programs and languages.

PART 1 → PART 2

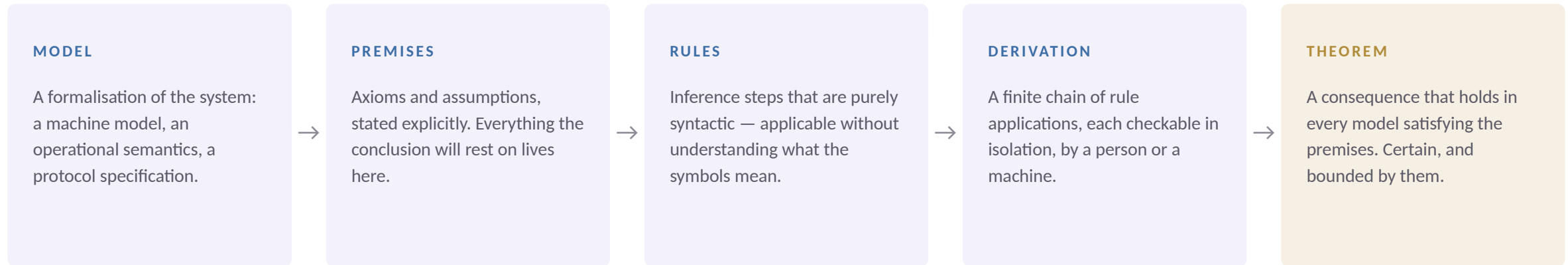
From the shape of an inference to the machinery that runs it

Deduction only guarantees anything inside a formal system. What does such a system consist of, what does it promise, and where does the promise run out?

Next: deduction in computing science — formal systems, soundness, and their limits.

Deduction: From Model to Consequence

Deriving a particular consequence from a general rule by steps that cannot introduce error — inside a model you chose.



Mechanical by construction

Because the rules are syntactic, a derivation can be checked without judgement. This is precisely what makes proof assistants — Coq, Isabelle, Lean — possible at all.

The model is the contribution

Choosing what to formalise, and at what level of detail, is the creative act. The derivation that follows is comparatively routine.

Where CS differs from mathematics

A CS theorem usually concerns an artifact that also exists physically. The gap between the formal model and the running system is a permanent part of the argument.

Anatomy of a Formal System

Five components. The last two are the ones a research claim actually trades on.

1

Syntax

The well-formed formulas — what can be written down at all. Grammar, not meaning.

2

Semantics

What the formulas mean: the models, states, or executions in which a formula is true. Written $\models$.

3

Inference rules

The sanctioned moves from formulas to formulas. Purely syntactic, so a machine can apply and check them. Written $\vdash$.

4

Soundness

Everything provable is true: $\vdash \varphi$ implies $\models \varphi$. Without it, a proof means nothing — you can derive falsehoods.

5

Completeness

Everything true is provable: $\models \varphi$ implies $\vdash \varphi$. Without it, some truths are simply out of the system's reach.

YOU CANNOT ALWAYS HAVE BOTH

Gödel showed that any consistent system strong enough to express arithmetic cannot be both complete and able to prove its own consistency. In CS this is not a curiosity — it is why no automatic tool decides every interesting property of every program.

HOW TOOLS CHOOSE

Verifiers and static analysers are usually sound but incomplete: they may raise false alarms on correct programs, but never stay silent on a real error. Bug-finders often invert the trade — complete in practice, unsound in principle. A tool paper must say which it is.

Deduction at Work in Computing Science

Six research areas whose central results are derived, not measured. Each is a formal system with its own syntax, semantics, and rules.

Type systems

Progress and preservation together give type soundness: a well-typed program cannot get stuck. The proof is structural induction over typing derivations.

Program logics

Hoare logic and separation logic derive $\{P\} C \{Q\}$ triples from axioms and rules. The basis of Dafny, Why3, Frama-C, and Iris.

Model checking

Exhaustive deduction over a finite state space: a temporal-logic property is decided for every reachable state, not sampled on some of them.

Complexity theory

Bounds, hierarchies and separations derived inside a machine model. Lecture 2's $\Omega(n \log n)$ argument was one of these.

Cryptographic reductions

Security by deduction from an assumption: breaking this scheme would solve a problem believed hard. The guarantee is conditional, and says so.

Distributed protocols

Safety and liveness of consensus algorithms — Paxos, Raft, PBFT — specified and machine-checked in TLA+, Coq, or Ivy.

Worked Example: Deriving a Hoare Triple

Claim under test: $\{n \geq 0\} \text{ SUM } \{ \text{sum} = n(n-1)/2 \}$ — the program computes the sum of $0 \dots n-1$, for every n .

```
sum := 0
i := 0
while i < n do
  sum := sum + i
  i := i + 1
```

THE LOOP INVARIANT

$I \equiv \text{sum} = i(i-1)/2 \wedge 0 \leq i \leq n$

Stated once, it discharges every obligation opposite.

Lecture 2 argued an invariant in prose; Hoare logic is the calculus that makes the same argument mechanical, and therefore machine-checkable. Termination is a separate obligation: i strictly increases and is bounded above by n .

1

Establish I — the assignment axiom

The rule $\{Q[e/x]\} x := e \{Q\}$ is applied backwards through $i := 0$ and $\text{sum} := 0$, yielding the precondition $0 = 0 \wedge 0 \leq 0 \leq n$. The given $n \geq 0$ implies it, so I holds when the loop is first reached.

2

Preserve I — the while rule

From $\{I \wedge B\} \text{ body } \{I\}$ the rule infers $\{I\} \text{ while } B \text{ do body } \{I \wedge \neg B\}$. The single obligation is that the body re-establishes I , assuming I and $i < n$ held on entry.

3

Discharge the body obligation

After $\text{sum} := \text{sum} + i$ we have $\text{sum} = i(i-1)/2 + i = i(i+1)/2$. After $i := i + 1$ the counter is $i+1$, and $i(i+1)/2$ is exactly $(i+1)((i+1)-1)/2$ — which is I at the new value of i .

4

Conclude — the rule of consequence

On exit $I \wedge \neg(i < n)$ gives $\text{sum} = i(i-1)/2 \wedge i = n$, and weakening that yields $\text{sum} = n(n-1)/2$. The triple is derived, for every n at once.

The Limits of Deduction

Four boundaries worth stating in your own limitations section, before a reviewer states them for you.

LIMIT 1

Premises, not truth

Soundness guarantees the conclusion given the axioms — and nothing about the axioms.

IN PRACTICE

A verified system is correct with respect to its specification. A wrong specification is verified just as cheerfully as a right one.

LIMIT 2

Undecidability

No complete automatic procedure exists for non-trivial semantic properties of programs.

IN PRACTICE

Rice's theorem, met again from Lecture 2. Every analyser must sacrifice completeness, soundness, or termination — and should say which.

LIMIT 3

State explosion

Exhaustive methods are exhaustive only over spaces small enough to exhaust.

IN PRACTICE

Model-checking state grows exponentially in concurrent components. Abstraction restores tractability and reintroduces approximation.

LIMIT 4

No new content

Deduction unpacks what the premises entail. It cannot propose the premises.

IN PRACTICE

It will not tell you which axioms to adopt, which model to formalise, or which hypothesis is worth testing. That is abduction's work — Part 4.

None of these makes deduction weak. They make its scope precise — and a precisely scoped guarantee is worth more to a reader than an unbounded one.

Induction: From Instances to Generalisation

Reasoning from observed instances to a rule that covers unobserved ones. The conclusion is supported by the premises, never entailed by them.

INSTANCES

A finite sample: training examples, benchmark runs, mined repositories, study participants.

generalisation



the leap

RULE

A hypothesis, model, or classifier that applies to cases nobody has yet observed.

THE ASYMMETRY

Deduction tells you what follows.

Induction tells you what to expect.

Only the second can be wrong while every step was correct.

This is why inductive claims must carry qualifiers and deductive ones must carry premises. Each names the place where the reasoning could fail.

- **Ampliative, and therefore fallible.**

The rule asserts something about cases the sample never contained. That surplus is exactly what makes it useful, and exactly what makes it defeasible.

- **Lecture 2's empirical mode, seen from the inside.**

Everything said there about sampling, controls and effect size is machinery for making one specific inductive leap defensible.

- **In CS the leap is usually automated.**

A learning algorithm performs the induction. We therefore reason about the algorithm's guarantees rather than about each individual inference it makes.

Where Induction Lives in Computing Science

Four research areas built entirely on the inductive leap. In each, the interesting question is what licenses the generalisation.

AREA 1

Machine learning

A learner is literally an inductive engine: it maps a finite sample to a function over an infinite domain.

WHERE THE LEAP HIDES

The generalisation claim is the contribution. Test accuracy is the sample; deployment accuracy is the population.

AREA 2

Empirical software engineering

Rules induced from observed projects: defect prediction, effort estimation, developer studies.

WHERE THE LEAP HIDES

The population is “software projects”. The sample is usually “public repositories that happen to be mineable”.

AREA 3

Performance modelling

Regression models and scaling laws fitted to measured runs, then extrapolated past the measured range.

WHERE THE LEAP HIDES

Extrapolation is induction at its most exposed: the fitted curve has no evidence at all where the claim matters most.

AREA 4

Algorithm engineering

Heuristics tuned on instance families and reported as good on “typical” inputs.

WHERE THE LEAP HIDES

Average-case and “works well in practice” claims are inductive, even when made about an object that also has theorems.

In all four, the honest question is the same: what is the population, and what makes this particular sample stand in for it?

Inductive Bias and the No-Free-Lunch Theorem

Why assumption-free learning is not merely difficult but impossible — and why that is good news for your methodology section.

INDUCTIVE BIAS

The assumptions a learner uses to prefer one generalisation over the others equally consistent with the data.

Any finite sample is consistent with infinitely many rules. Something other than the data must choose between them — the hypothesis class itself, a smoothness or sparsity prior, convolutional locality, a preference for shorter descriptions.

Every learner has one. The only question is whether the paper states it.

NO FREE LUNCH

Averaged over all possible target functions, every learning algorithm has the same expected off-training-set error.

Wolpert and Macready's result says superiority is never universal: an algorithm that does better than another on one class of problems must do worse on some other class. There is no best learner, only a best-matched one.

So “our method is better” always means “better on problems shaped like these”.

Bias is not a defect to be engineered away — it is the precondition for generalising at all. Which makes stating it a methodological obligation, not an admission: an unstated bias is an unstated premise, and Part 2 already established what that does to a claim.

Worked Example: Induction, Made Quantitative

Claim under test: “With $m \geq (1/\epsilon)(\ln |H| + \ln(1/\delta))$ samples, any hypothesis consistent with the data has true error $\leq \epsilon$, with probability $\geq 1 - \delta$.” — Valiant's PAC framework, 1984

1

Set up

An unknown distribution D over inputs, an unknown target concept, a finite hypothesis class H fixed in advance, and m examples drawn independently from D .

2

Define “bad”

Call h bad if its true error on D exceeds ϵ . A single bad h agrees with one random example with probability at most $1 - \epsilon$, so it survives all m with probability $\leq (1 - \epsilon)^m$.

3

Union bound

There are at most $|H|$ bad hypotheses, so the chance that any of them survives all m examples is at most $|H|(1 - \epsilon)^m \leq |H|e^{(-\epsilon m)}$.

4

Solve for m

Require $|H|e^{(-\epsilon m)} \leq \delta$. Rearranging gives $m \geq (1/\epsilon)(\ln |H| + \ln(1/\delta))$ — a sample size that makes the inductive leap quantitatively safe.

WHY THIS SLIDE SITS IN BOTH PARTS

Every line of the argument is deductive — yet the theorem is about induction. Learning theory is precisely this: deduction applied to the reliability of inductive procedures, converting “it seems to generalise” into a bounded quantity.

AND WHERE IT STOPS

Read the premises: examples independent and identically distributed, H fixed before seeing data, some hypothesis in H consistent with it. Modern practice violates all three routinely — which is why the bound is a way of thinking, not a certificate.

The Limits of Induction

Four ways a correctly executed generalisation still ends up false. Each has a standard name and a standard defence.

LIMIT 1

Sampling bias

The population you sampled is not the population your claim is about.

IN PRACTICE

Defence: describe the sampling frame explicitly, and qualify the claim down to it. GitHub is not software; ImageNet is not vision.

LIMIT 2

Overfitting

A rule that fits the sample perfectly and the population poorly.

IN PRACTICE

Defence: held-out data untouched during development, and reporting how many configurations were tried before this one.

LIMIT 3

Distribution shift

The i.i.d. premise that licensed the generalisation fails after deployment.

IN PRACTICE

Defence: measure on data from a genuinely later period or a different source, and report the degradation rather than the best case.

LIMIT 4

Goodhart's law

Once a benchmark becomes the target, it stops being a good measure.

IN PRACTICE

Defence: report on more than one benchmark, and treat saturation on a leaderboard as evidence about the benchmark, not the field.

All four are failures of the population, not of the arithmetic. That is why Lecture 2 put threats to validity, and not statistical technique, at the centre of empirical review.

Abduction: Inference to the Best Explanation

From a surprising observation to the hypothesis that, if true, would make the observation a matter of course.

PEIRCE'S SCHEMA

The surprising fact C is observed. · But if H were true, C would follow as a matter of course.
Hence, there is reason to suspect that H is true.

- **It is the only direction that creates hypotheses.**

Deduction and induction can only operate on candidates already on the table. Everything either of them ever works on was put there by an abduction.

- **It fails in one specific way.**

The conclusion is only as good as the field of rivals you actually considered. Abduction over a single candidate is not inference — it is a guess with a citation attached.

- **In CS it is daily practice, rarely named.**

Debugging is abduction: symptom to cause. So are fault localisation, root-cause analysis, performance triage — and framing a research problem, which is Unit I's whole subject.

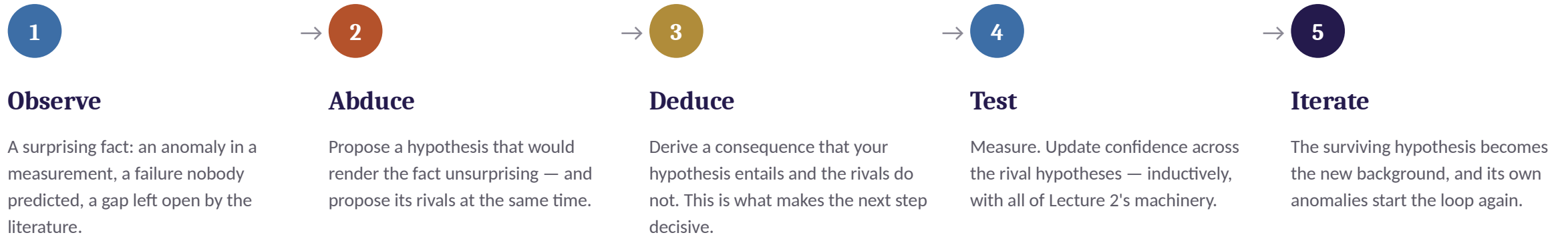
THE STANDARD OF EVIDENCE

“Best explanation” is meaningful only relative to the alternatives you listed.

So an abductive step is defended by enumeration, not by eloquence: name the rival explanations, say what each would predict, and show which predictions the observation rules out. A related-work section that reads as antithesis — Lecture 1's point — is doing exactly this job.

The Research Cycle: Abduce, Deduce, Test

The three directions are not alternatives to choose between. They are stages of one loop, and each does work the others cannot.



WHY STEP 3 IS THE ONE PAPERS SKIP

It is easy to observe, easy to propose an explanation, and easy to collect data that is consistent with it. What is hard — and what separates a study from an anecdote — is deriving a prediction that the rival explanations would not also make. Evidence consistent with every hypothesis on the table discriminates between none of them, however much of it you gather.

What Counts as a Theoretical Model

A theoretical model is a deliberate simplification, adopted so that deduction becomes possible. David Marr's three levels are the standard way to say which simplification you have made.

LEVEL 1

Computational

What problem is being solved, and why is solving it appropriate?

The specification, independent of any method: the input-output relation and the constraints it must respect.

Sorting: return a permutation of the input in non-decreasing order.

LEVEL 2

Algorithmic

What representation, and what process transforms input into output?

The data structures and the procedure — still abstract, still independent of any particular machine.

Mergesort over arrays: divide, recurse, merge; $\Theta(n \log n)$ comparisons.

LEVEL 3

Implementational

How is the process physically realised?

The substrate: instruction set, memory hierarchy, cache-line behaviour, compiler, hardware — or, in Marr's original setting, neurons.

A cache-aware merge in C, tuned for L2 residency on a specific x86 generation.

A great many confused CS papers confuse the levels: a claim proved at level 1 defended with level-3 measurements, or a level-3 optimisation reported as a level-2 advance. Naming your level is half of naming your contribution.

Models of Computation as Deductive Substrates

Every cost claim and every impossibility result is stated inside one of these. The model is a premise — usually the load-bearing one.

Turing machine

Unbounded tape, finite control.

Defines computability itself; the reference model for decidability and reductions.

λ -calculus

Functions as the primitive; substitution as the only rule.

Substrate of functional languages, type theory, and proof assistants.

RAM model

Unit-cost access to any memory cell.

The default for algorithm analysis — and the premise Lecture 2 attacked.

PRAM

Many processors, synchronous shared memory.

Parallel algorithm design and lower bounds, abstracting away communication cost.

External-memory model

Counts block transfers, not instructions; cache-oblivious variants too.

Predicts real performance once data exceeds cache, where RAM misleads.

Finite automata

Finite state, no auxiliary memory.

Lexing, regular languages, protocols, and the substrate for model checking.

Process calculi

Concurrent agents exchanging messages — CSP, π -calculus, actors.

Reasoning about concurrency, deadlock, and distributed protocol correctness.

Probabilistic models

Randomised steps; Markov chains; classes such as BPP.

Randomised algorithms, average-case analysis, and probabilistic model checking.

Choosing a model trades fidelity against tractability: the more faithfully it prices real hardware, the harder it becomes to prove anything in it. State which one you chose, and why.

Matching Reasoning to the Research Question

Read the question you are actually asking, then read across. The last column is where the review will land.

THE QUESTION YOU ARE ASKING	DOMINANT DIRECTION	WHAT LICENSES IT	WHAT A REVIEWER ATTACKS
“Why does this system fail under X?”	Abduction	Rival explanations enumerated and ruled out	The explanation you never considered
“Does this property always hold?”	Deduction	A formal model with premises stated	The premise that fails in deployment
“Does this method generalise?”	Induction	Sampling design, held-out data, a stated bias	The distribution you never sampled
“What does this algorithm cost?”	Deduction, in a model	The machine model the analysis assumes	Whether that model prices real hardware
“Which account of this data is right?”	Abduction, then deduction	A prediction that separates the rivals	Two hypotheses predicting the same thing

Most methodological objections reduce to a mismatch in this table — a question asked in one direction and answered in another.

Reasoning Failures to Watch For

Lecture 1 catalogued argumentative fallacies and Lecture 2 evidential ones. These are their inferential cousins — errors in the direction of travel itself.

Affirming the consequent

“Our theory predicts X. We observed X. Therefore the theory is true.” X may follow from several theories; observing it eliminates none of them on its own.

Deductive overreach

Treating a theorem's conclusion as a claim about a running system, when the model's premises were never checked against that system.

The convenience sample

Generalising from whatever data was easiest to collect — public repositories, undergraduate participants, one hardware generation — as though it were the population.

Stopping at the first explanation

An abduction that never enumerated rivals, so “best explanation” silently means “the only one that occurred to us.”

Confusing the two inductions

Presenting a proof by mathematical induction as empirical support, or an empirical trend as though it had been proved. Different obligations, identical vocabulary.

Model laundering

Dressing an inductive guess in formal notation so that it inherits deduction's authority without acquiring deduction's guarantees.

A Checklist for Your Own Reasoning

- ✓ **Name the direction.** For each central step: deductive, inductive, or abductive? A paper that cannot answer this for its main claim has not yet decided what it is arguing.
- ✓ **Expose the premises, whichever direction you took.** Deduction: which axioms must hold for the theorem to bite? Induction: what population was sampled? Abduction: which rivals were on the table?
- ✓ **Enumerate before you abduce.** “Best explanation” is empty until at least two candidates exist and you have said what would distinguish them.
- ✓ **Keep the two inductions apart.** Mathematical and structural induction prove. Empirical induction supports. Never let one word carry both obligations in the same paragraph.
- ✓ **State your model and its level.** Which model of computation licenses your cost claims, and which of Marr’s levels is your contribution at? Both belong in the paper, not in your head.
- ✓ **Say what would falsify it.** A deductive claim falls to a counter-example or a false premise; an inductive one to a shifted distribution; an abductive one to a better rival.

Discussion

In pairs, ten minutes. Bring one answer back for class critique.

- 1 Take a theorem from your own area and write out its premises as a numbered list. Which premise is most likely to be false in a system you have actually run — and what would that do to the conclusion?
- 2 Find a machine-learning or empirical software-engineering paper. What is its inductive bias, and where in the paper is it stated? If it is not stated, what would you have to know to reproduce the result?
- 3 Reconstruct a bug you have debugged as an abduction. Which hypotheses did you hold at once, and what observation finally discriminated between them?
- 4 Take a performance claim from any paper. Which model of computation does its cost argument assume, and does that model price the hardware you would run it on?

Key Takeaways

- 1 Deduction preserves truth but adds no content; induction adds content but cannot preserve truth; abduction generates the hypotheses on which the other two go to work. Research needs all three, in that cycle.
- 2 Mathematical and structural induction are deductive despite the name. Confusing them with empirical induction lets a hope be read as a guarantee — the most consequential terminological slip in CS writing.
- 3 Deduction in CS happens inside a formal system, and delivers exactly the strength of its premises: soundness, completeness, and the chosen model are all part of the claim.
- 4 Induction is only possible because of inductive bias. No-free-lunch says an assumption-free learner cannot generalise; PAC theory bounds how much data a stated bias needs. So state the bias.
- 5 A theoretical model is a deliberate simplification adopted to make deduction possible. Name the model, name its level in Marr's sense, and name where its predictions stop.

Further Reading

Peirce, C. S. “Deduction, Induction, and Hypothesis.” Popular Science Monthly, 1878; Collected Papers, Vol. 2.

Hoare, C. A. R. “An Axiomatic Basis for Computer Programming.” Communications of the ACM, 1969.

Pierce, B. C. Types and Programming Languages. MIT Press, 2002. — progress, preservation, and structural induction.

Winskel, G. The Formal Semantics of Programming Languages. MIT Press, 1993.

Clarke, E. M., Grumberg, O. & Peled, D. Model Checking. MIT Press, 1999. — exhaustive deduction and state explosion.

Valiant, L. G. “A Theory of the Learnable.” Communications of the ACM, 1984. — the PAC framework.

Shalev-Shwartz, S. & Ben-David, S. Understanding Machine Learning: From Theory to Algorithms. Cambridge University Press, 2014.

Wolpert, D. H. & Macready, W. G. “No Free Lunch Theorems for Optimization.” IEEE Trans. Evolutionary Computation, 1997.

Marr, D. Vision. Freeman, 1982. — the three levels of analysis, Chapter 1.